Design Patterns Course

Learn, Build, Adapt

Session #6

2024

Irina Kamalova

VP Lead Software Engineer @ JPMC
ex-Revolut & ex-Yandex
Built systems to support 45M+ clients

Refreshing

- What **Facade Pattern** is useful for?
- What **Template Method Pattern** is useful for?

06.1

Iterator

Iterator

…Don't you think **you meet** this pattern **everyday**?

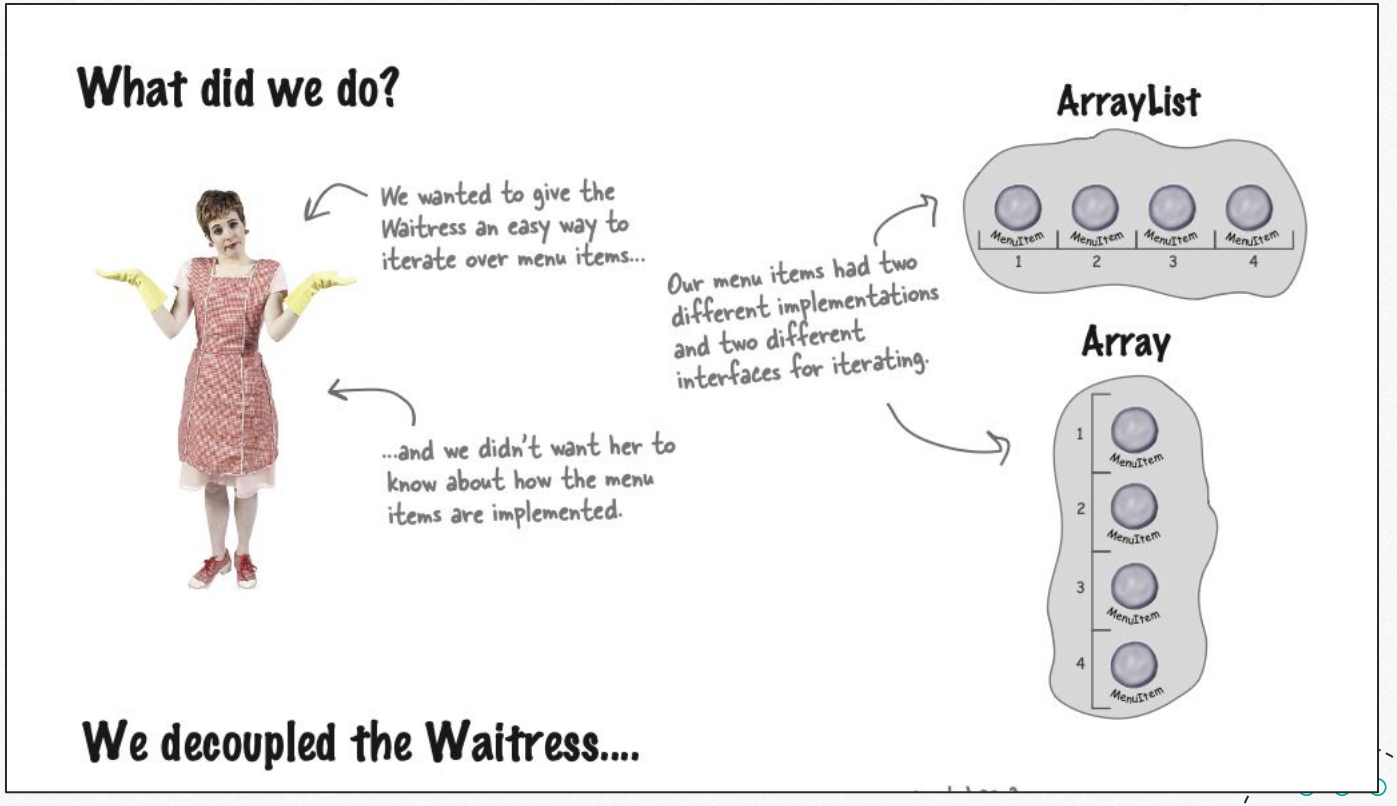
Behavioral

Behavioral Patterns

- Chain of Responsibility
- Command
- → Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

We **create** a **pattern** for some **behavior** from scratch.

Example


What did we do?
We wanted to give the Waitress an easy way to iterate over menu items...
...and we didn't want her to know about how the menu items are implemented.
Our menu items had two different implementations and two different interfaces for iterating.
ArrayList
MenuItem
1
2
3
4
Array
MenuItem
1
2
3
4
We decoupled the Waitress....

Iterator

The Iterator Pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

Let's dive deeper...

What is wrong with the next code?

```
public MenuItem[] getMenuItems() {
    return menuItems;
}
```

```
public ArrayList<MenuItem> getMenuItems() {
    return menuItems;
}
```

Lou has

Another way to iterator

```java
public static void main(String[] args) {
    MenuItem[] breakfastMenu = BreakfastMenu.getAllMenuItems();
    ArrayList<MenuItem> diningMenu = DiningMenu.getAllMenuItems();

    ArrayList<MenuItem> menu = new ArrayList<>();
    menu.addAll(asList(breakfastMenu));
    menu.addAll(diningMenu);

    for (MenuItem menuItem : menu) {
        System.out.println(menuItem);
    }
}
```

Let's dive deeper

```java
class Scratch {
    public static void main(String[] args) {
        var list = new ArrayList<String>();
        list.add("A");
        list.add("B");
        list.add("C");

        for (String s : list) {
            System.out.println(s);
            list.remove(s);
        }
    }
}
```

What is going to be the result?

Let's dive deeper

```
A
Exception in thread "main" java.util.ConcurrentModificationException Create breakpoint
    at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1095)
    at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1049)
    at Scratch.main(scratch.java:14)
```

Let's dive deeper

```java
class Scratch {
    public static void main(String[] args) {
        var list = new ArrayList<String>();
        list.add("A");
        list.add("B");
        list.add("C");

        for (String s : list) {
            System.out.println(s);
            int indexOfS = list.indexOf(s);
            System.out.println(indexOfS);
            list.add(indexOfS, s);
        }
    }
}
```

What is going to be the result?

Let's dive deeper

```
A
0
Exception in thread "main" java.util.ConcurrentModificationException Create breakpoint
    at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1095)
    at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1049)
    at Scratch.main(scratch.java:10)
```

Let's consult with Raccoon

https://refactoring.guru/design-patterns/iterator

Python

The iterator pattern is implemented in Python using two main components: the **iterator** and the **iterable**. Python provides built-in support for this pattern through the use of the `iter()` and `next()` functions, along with special methods `__iter__()` and `__next__()`.

Here's how it works:

Components of the Iterator Pattern in Python

1. **Iterable**:
   - An object that has an `__iter__()` method, which returns an iterator.
   - Examples include lists, tuples, dictionaries, and custom iterable classes.
2. **Iterator**:
   - An object that implements the `__next__()` method to return the next value and raises `StopIteration` when there are no more values.
   - An iterator object must also implement `__iter__()` (usually returning `self`) to be compatible with the `iter()` function.

Python's Built-in Iterators and Generators

Python simplifies iterator creation with **generators**, which use `yield` to create an iterator automatically.

```python
def my_generator():
    for i in range(1, 5):
        yield i

# Using the generator
for value in my_generator():
    print(value)  # Outputs: 1, 2, 3, 4
```

Are you using data generators for testing purposes?

Python: Custom Implementation

Custom Logic for Even Numbers Only

```python
class EvenNumbers:
    def __init__(self, start, end):
        self.current = start if start % 2 == 0 else start + 1
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        result = self.current
        self.current += 2
        return result

# Using the custom iterator
for num in EvenNumbers(3, 15):
    print(num)  # Outputs: 4, 6, 8, 10, 12, 14
```

Python: Custom Implementation

Lazy Fibonacci Sequence

Implementing a custom iterator allows you to generate elements on demand rather than storing them all in memory. This is especially useful when working with large datasets or infinite sequences.

```python
class Fibonacci:
    def __init__(self):
        self.a, self.b = 0, 1

    def __iter__(self):
        return self

    def __next__(self):
        self.a, self.b = self.b, self.a + self.b
        return self.a

# Using the iterator (e.g., first 10 Fibonacci numbers)
fib = Fibonacci()
for i in range(10):
    print(next(fib))  # Outputs: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55
```

Python: support for multiple iteration styles

Custom iterators can provide different ways of iterating over the same data by using multiple iterators. For example, you could create one iterator for forward traversal and another for reverse traversal.

```python
class BidirectionalRange:
    def __init__(self, start, end):
        self.data = list(range(start, end))

    def __iter__(self):
        return iter(self.data)

    def reverse(self):
        return iter(self.data[::-1])

# Using the bidirectional iterator
br = BidirectionalRange(1, 5)
print(list(br))  # Outputs: [1, 2, 3, 4]
print(list(br.reverse()))  # Outputs: [4, 3, 2, 1]
```

System Design World

Cursor-based Pagination

- **What it is:** Commonly used in APIs and databases, cursor-based pagination allows clients to fetch data in chunks (e.g., "next 100 items") without knowing the structure of the dataset.
- **How it's similar:** Just as an iterator abstracts the process of traversing a collection, a cursor abstracts sequential traversal of a dataset while maintaining state between calls.
- **Example:**
  - **SQL Query:** `SELECT * FROM orders WHERE id > last_id LIMIT 100;`
  - **API Example:** A RESTful or GraphQL API that supports cursors, such as `GET /items?cursor=abc123`.

Have you worked with pagination?

System Design World

Workflow Systems (Chained Execution)

- **What it is:** Workflow orchestration systems like Apache Airflow or AWS Step Functions execute tasks in a defined sequence, with each step acting on the output of the previous one.
- **How it's similar:** Workflows can resemble iterators when tasks are designed to consume items one at a time from a queue or data source.
- **Example:** A task iteratively processes batches of files or data objects.

Have you worked with Airflow or similar framework?

Iterator

…Don't you think **you meet** this pattern **everyday**?

06.2

Composite

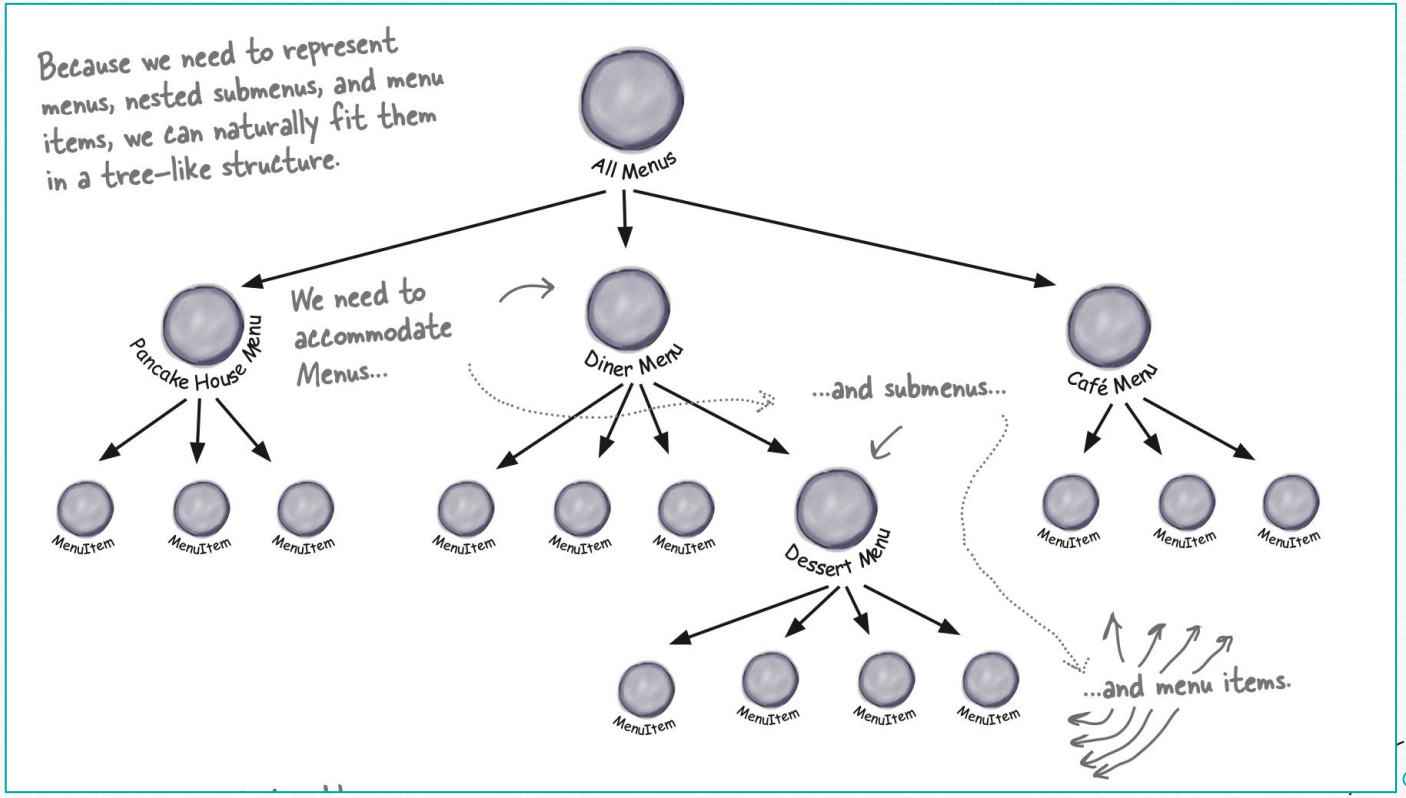
Structural

Structural Patterns

- Adapter
- Bridge
- → Composite
- Decorator
- Facade
- Flyweight
- Proxy

We don’t change the behavior of original action/actor/entity, but we **create** a **pattern** for the **structure** to add new behaviour or change the behavior by adding a new layer.

Example

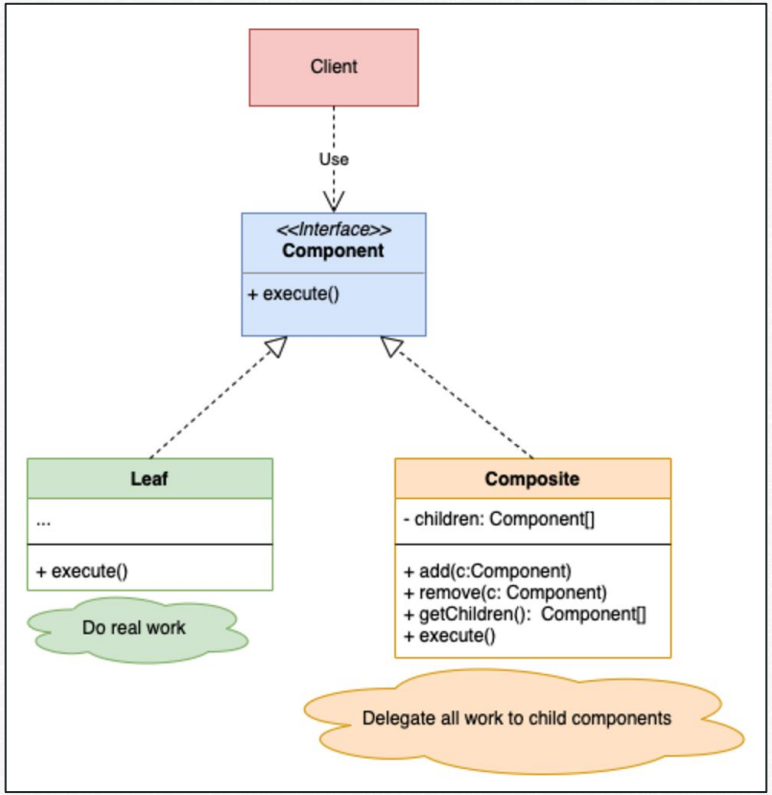

Because we need to represent menus, nested submenus, and menu items, we can naturally fit them in a tree-like structure.
All Menus
Pancake House Menu
We need to accommodate Menus...
Diner Menu
...and submenus...
Café Menu
MenuItem
MenuItem
MenuItem
MenuItem
MenuItem
MenuItem
Dessert Menu
MenuItem
MenuItem
MenuItem
MenuItem
MenuItem
MenuItem
MenuItem
...and menu items.

The Composite Pattern

The Composite Pattern allows you to compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

Class diagram

Source



More examples (Java implementation)

Workflow Systems (Chained Execution)

- **File system:** https://www.learncsdesign.com/learn-the-composite-design-pattern/
- **Render HTML DOM tree:** https://medium.com/@kamresh485/composite-pattern-in-c-d9b72997e5aa
- **Menu:** https://dev.to/superviz/design-pattern-6-composite-pattern-2k1l#:~:text=Menu%3A%20This%20class%20represents%20composite,%2C%20and%20a%20sub%2Dmenu.
- **Another file system:** https://medium.com/@ngneha090/beginners-guide-to-composite-design-pattern-java-f9ef644e3425

Let's consult with Raccoon

https://refactoring.guru/design-patterns/composite

Let's consult with Raccoon

Composite and **Decorator** have similar structure diagrams since both rely on recursive composition to organize an open-ended number of objects.

A *Decorator* is like a *Composite* but only has one child component. There's another significant difference: *Decorator* adds additional responsibilities to the wrapped object, while *Composite* just "sums up" its children's results.

However, the patterns can also cooperate: you can use *Decorator* to extend the behavior of a specific object in the *Composite* tree.

Python Example

```python
# Component
class FileSystemComponent(ABC):
    """Abstract base class for file system components."""

    @abstractmethod
    def show_details(self, indent=0):
        pass
```

```python
# Leaf
class File(FileSystemComponent):
    """Represents a file in the file system."""

    def __init__(self, name, size):
        self.name = name
        self.size = size

    def show_details(self, indent=0):
        print(" " * indent + f"File: {self.name} ({self.size} KB)")
```

Python Example

```python
# Composite
class Directory(FileSystemComponent):
    """Represents a directory in the file system."""

    def __init__(self, name):
        self.name = name
        self.contents = []  # To hold child FileSystemComponents

    def add(self, component):
        self.contents.append(component)

    def remove(self, component):
        self.contents.remove(component)

    def show_details(self, indent=0):
        print(" " * indent + f"Directory: {self.name}")
        for component in self.contents:
            component.show_details(indent + 4)
```

Python Example

```python
# Client code
if __name__ == "__main__":
    # Create leaf nodes (files)
    file1 = File("file1.txt", 10)
    file2 = File("file2.txt", 20)
    file3 = File("file3.txt", 15)

    # Create composite nodes (directories)
    dir1 = Directory("dir1")
    dir2 = Directory("dir2")
    root = Directory("root")

    # Build the hierarchy
    dir1.add(file1)
    dir1.add(file2)
    dir2.add(file3)
    root.add(dir1)
    root.add(dir2)

    # Show details of the entire hierarchy
    root.show_details()
```

```
Directory: root
    Directory: dir1
        File: file1.txt (10 KB)
        File: file2.txt (20 KB)
    Directory: dir2
        File: file3.txt (15 KB)
```

Python: *Duck typing*

What is Duck Typing?

Duck typing is a concept in dynamically typed programming languages (like Python), where the type or class of an object is determined by its **behavior** (methods and properties) rather than its explicit type. The name comes from the saying:

> **"If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck."**

In practice, this means that Python doesn't enforce explicit interfaces or type hierarchies. Instead, it assumes that if an object implements the necessary methods and properties required by the operation, it can be used regardless of its actual type.

Duck Typing example for a File Storage System

```python
class File:
    """Represents a file in the file storage system."""

    def __init__(self, name: str, size: int):
        self.name = name
        self.size = size

    def show_details(self, indent: int = 0) -> None:
        print(" " * indent + f"File: {self.name} ({self.size} KB)")
```

Duck Typing example for a File Storage System

```python
class Directory:
    """Represents a directory in the file storage system."""

    def __init__(self, name: str):
        self.name = name
        self.children = []  # Can hold File or Directory objects

    def add(self, item) -> None:
        self.children.append(item)

    def show_details(self, indent: int = 0) -> None:
        print(" " * indent + f"Directory: {self.name}")
        for child in self.children:
            child.show_details(indent + 4)
```

Duck Typing example for a File Storage System

```python
if __name__ == "__main__":
    # Create files
    file1 = File("file1.txt", 10)
    file2 = File("file2.txt", 20)
    file3 = File("file3.txt", 15)

    # Create directories
    dir1 = Directory("dir1")
    dir2 = Directory("dir2")
    root = Directory("root")

    # Build hierarchy
    dir1.add(file1)
    dir1.add(file2)
    dir2.add(file3)
    root.add(dir1)
    root.add(dir2)

    # Display the file system structure
    root.show_details()
```

```
Directory: root
    Directory: dir1
        File: file1.txt (10 KB)
        File: file2.txt (20 KB)
    Directory: dir2
        File: file3.txt (15 KB)
```

Duck Typing example for a File Storage System

Pros and Cons

Pros

1. **Simplicity**: No boilerplate code for abstract base classes or interfaces.
2. **Flexibility**: Any object that implements a `show_details` method can be added to a directory.
3. **Readability**: Straightforward design that's easy to understand.

Cons

1. **No Static Checking**: Tools like `mypy` can't enforce that objects have the required `show_details` method, which could lead to runtime errors if improperly used.
2. **Scalability**: For large systems, the lack of a clear interface might lead to inconsistencies.
3. **Extensibility**: Without a base class or common interface, adding behaviors shared across `File` and `Directory` requires duplication or additional helper functions.

LSP in Python and Duck Typing

In Python, inheritance isn't a requirement for substitutability. Instead, **duck typing** shifts the focus to whether objects exhibit the expected behavior (methods and properties). This makes LSP less about inheritance and more about **contracts of behavior**.

Why LSP still applies?

The core of LSP is ensuring substitutability. While Python doesn't enforce inheritance, the principle remains relevant:

- Objects substitutable for one another must maintain the **behavioral expectations** of the context in which they're used.
- Behavioral consistency is what matters, not the mechanism (inheritance vs. duck typing).

Substitutability without inheritance is risky?

No compile-time checks:

- In statically-typed languages with inheritance, the compiler enforces that derived classes meet the interface of the base class.
- In Python, there’s no mechanism to enforce this; you only discover problems at runtime.

Accidental typing errors:

- A method might exist but with a typo or different parameters, causing errors at runtime instead of compile time.

Implicit contracts:

- Duck typing relies on implicit behavioral contracts, which are error-prone and require careful documentation and testing.
- Developers must manually ensure that objects conform to expectations, which isn't guaranteed.

Popular languages that support Duck Typing

- Python
- Rust
- Go
- JavaScript
- PHP
- Perl

Your Journey

01

Introduction
&
Strategy pattern

02

Observer
&
Decorator

03

Creational
Patterns

04

Command
Adapter

05

Facade &
Template Method

06

Iterator &
Composite

07

...

08

...

Your Journey

09

...

10

...

11

...

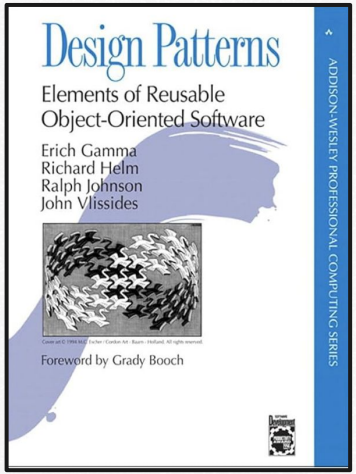
Resources

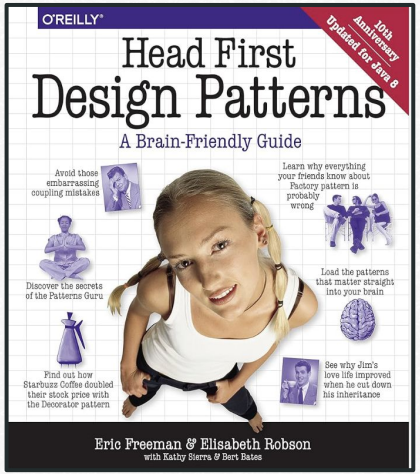
Books:

- Classic GOF Pattern
- Racoon Book
- Head First Design Patterns

Examples:

- Presentation (in Slack)





Thank you!